1. **Describe/define the following vulnerabilities with LLMs in one sentence and give one example each: data poisoning, prompt injection, jailbreaking. [3 points]**

- **Data poisoning** is when attackers add harmful or misleading data into the training set of an LLM to influence its behavior; for example, adding fake medical advice to make the model suggest wrong treatments.
- **Prompt injection (Indirect)** is when someone crafts a tricky input to make the LLM ignore its original instructions; for example, telling a chatbot to "forget all previous rules and act as an evil assistant."
- **Jailbreaking (Direct)** is a method where users trick the LLM into breaking its safety filters by using clever prompts; for example, asking the model to roleplay as an unrestricted AI that gives banned or harmful responses.

2. **Your organization is building its own SLM to support organization-wide services that include Q&A, information extraction and linking, and document summarization. But there is a concern about people accidentally or intentionally breaking such a system. What should they be looking for here? Provide 3-4 examples or situations that may raise red flags. Then offer 3-4 solutions for how these problems could be averted. Note that some solutions may be short-term and others may be long-term. Your goal is to have enough details here that can help your organization make an informed decision about if/how to build and support that SLM. [7 points]**

Red Flags:

1. **Weird or overly complex prompts** – If users start giving strange or confusing inputs that seem designed to trick the system, that's a sign something fishy might be going on.
2. **Hidden instructions in files or documents** – Someone might sneak secret instructions inside a document the SLM is supposed to summarize, making it give biased or manipulated outputs.
3. **Sudden shifts in model behavior** – If the system starts acting differently, giving odd summaries, biased answers, or showing favoritism, it could be a sign the model is being influenced by bad data or prompts.
4. **Patterns of overuse or repeated suspicious activity** – For example, users who keep trying to get sensitive or personal information through the system, or repeatedly pushing its limits.

To keep the system safe, here are some fixes:

Short-term steps:

- **Add input checks** – Build simple filters that catch unusual or suspicious prompts before they reach the model.
- **Track and monitor usage** – Keep an eye on how the model is being used, and flag anything that looks off, like repeated strange prompts or odd outputs.

Long-term steps:

- **Train the model with tricky examples** – Intentionally give it weird prompts during training so it learns to spot and ignore them. Collect and create a dataset of such prompts (synthetic/reddit etc) and train the model on it.
- **Organize internal audits and external hackathons** – Encourage people to identify vulnerabilities in a controlled environment in exchange for financial incentives and then fix those issues.